

Assignment 2-Case Study

Stage 3 Tutorial Activities

From Requirements to Domain Models

Week 6 | 60 minutes

Candidate Concepts

Candidate	Class?	Reason
Patient	Yes	Represents a patient whose information must be stored, searched and managed.
Practitioner	Yes	Represents a practitioner whose information and availability must be managed.
Appointment	Yes	Represents a booking between the patient and the practitioner and manages the status of said booking.
Name	No	An attribute of the patient or the practitioner.
Clinic	No	Case study only describes one clinic.
Database	No	A technical implementation concept rather than a domain entity.
Cancellation	No	Cancellation is a behaviour/state change of an Appointment, not a separate entity.
Status	No	Appointment status is an attribute/state of Appointment rather than a separate class.

CRC Cards

Patient

Responsibilities	Collaborators
Store patient details	Appointment
Provide patient information when searched	Appointment

Practitioner

Responsibilities	Collaborators
Maintain practitioner schedule	Appointment
Store practitioner details	Appointment

Appointment

Responsibilities	Collaborators
Link a patient to a practitioner	Patient and Practitioner
Manage appointment status	Patient and Practitioner

Relationship Reasoning

Patient to Appointment: which relationship and why?

A Patient has a one-to-many relationship with the Appointment. One patient can have zero or many appointments, while each appointment is associated with one patient.

Practitioner to Appointment: what multiplicity?

A Practitioner has a one-to-many relationship with the Appointment. One practitioner can have zero or many appointments, while each appointment is associated with one practitioner.

Should Appointment inherit from Patient?

No as appointments aren't a type of patient. They are separate domain concepts with different responsibilities. Appointments instead has an association with Patient and Practitioner.

Does Clinic need to own every object?

A Clinic class is unnecessary for the current scope. The case study describes a single clinic, and there is no identified requirement for Clinic-specific behaviour or relationships. Adding it would increase complexity without providing a required responsibility.

AI Model Critique

Critique AI proposals: PatientManager, PractitionerManager, AppointmentManager, ClinicController, NotificationManager, ScheduleEngine.

AI Proposal	Decision	Evidence
PatientManager	Rejected	Patient is already a domain class with responsibilities for storing and retrieving patient information. A separate manager class is not required by the current requirements and would add unnecessary complexity.
PractitionerManager	Rejected	Practitioner already manages practitioner information and availability. No requirement identifies a separate manager object.
AppointmentManager	Rejected	Appointment already represents bookings, status and cancellation. A separate manager is not supported by the current domain requirements.
ClinicController	Rejected	Clinic was not identified as a necessary domain class, and the requirements do not require a controller specifically associated

		with the clinic. This would be an implementation/architecture concept rather than a required domain concept.
NotificationManager	Rejected	There is no requirement for notifications, SMS reminders or email reminders. Adding this class would introduce unsupported functionality.
ScheduleEngine	Rejected	The requirements require viewing practitioner availability and preventing duplicate bookings, but do not require a separate scheduling engine. These responsibilities can currently be handled through Appointment and Practitioner without introducing another class.